

A FINOPS-BASED FRAMEWORK FOR CLOUD COST ALLOCATION, SHOWBACK, AND CHARGEBACK MANAGEMENT IN MULTI-CLOUD ENVIRONMENTS

A PROJECT REPORT

Submitted by

ABOORVAN S B	230701011
MOHAMMED SHAYAAN SHAIKH	230701189
SHAUN MACHADO K	230701309
SHAKTHI G	230701303

*in partial fulfillment for the award of the degree
of*

**BACHELOR OF ENGINEERING
IN
COMPUTER SCIENCE AND ENGINEERING**



RAJALAKSHMI ENGINEERING COLLEGE, CHENNAI

ANNA UNIVERSITY : CHENNAI 600 025

SEPTEMBER 2026

ANNA UNIVERSITY: CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this project report “A FINOPS-BASED FRAMEWORK FOR CLOUD COST ALLOCATION, SHOWBACK, AND CHARGEBACK MANAGEMENT IN MULTI-CLOUD ENVIRONMENTS” is the bonafide work of “ABOORVAN S B (230701011), MOHAMMED SHAYAAN SHAIKH (230701189), SHAUN MACHADO K (230701309) and SHAKTHI G (230701303)” who carried out the project work under my supervision.

SIGNATURE

Dr. J. Manoranjini

HEAD OF THE DEPARTMENT

Department of Computer Science and
Engineering
Rajalakshmi Engineering College
Chennai - 602 105

SIGNATURE

Mr. T.A Salman Latheef

SUPERVISOR

Assistant Professor
Department of Computer Science and
Engineering
Rajalakshmi Engineering College
Chennai - 602 105

Submitted for the Project Viva-Voce Examination held on

_____.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ABSTRACT

Cloud computing has become an essential part of modern organizational infrastructure due to its scalability, flexibility, and consumption-based pricing model. However, organizations using shared and multi-cloud environments face challenges in accurately identifying and allocating cloud expenditure across departments and projects. Consolidated billing, inconsistent resource ownership information, and provider-specific billing formats make cost attribution difficult. This project presents a FinOps-based framework for cloud cost allocation, showback, and chargeback management in multi-cloud environments. The proposed web-based platform provides a centralized workflow for importing, validating, and normalizing cloud billing data, mapping resources to departments and projects, and allocating infrastructure costs. The system follows the workflow of Billing Data, Validation, Cost Allocation, Showback/Chargeback, and Dashboard. It supports direct and shared cost allocation using configurable rules. The current implementation focuses on user access management, billing data import, and validation and normalization. Future enhancements include cloud API integration, budget forecasting, anomaly detection, and cost optimization. The proposed framework aims to improve cloud cost transparency, strengthen accountability, reduce manual processing, and support efficient utilization of cloud resources.

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman **Mr. S.MEGANATHAN, B.E, F.I.E.**, our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S.**, and our respected Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D.**, for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to **Dr. S.N. MURUGESAN, M.E., Ph.D.**, our beloved Principal for his kind support and facilities provided to complete our work in time.

We express our sincere thanks to **Dr.J. MANORANJINI**, Professor and Head of the Department of Computer Science and Engineering for his guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide, **Mr. T.A SALMAN LATHEEF**, Assistant Professor, Department of Computer Science and Engineering for his valuable guidance throughout the course of the project. We thank our Project Coordinator, **Dr. S. MADHUSUDHANAN**, Associate Professor, Department of Computer Science and Engineering for his useful tips during our review to build our project.

ABOORVAN S B

MOHAMMED SHAYAAN SHAIKH

SHAUN MACHADO K

SHAKTHI G

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iii
	ACKNOWLEDGEMENT	iv
	LIST OF TABLES	vii
	LIST OF FIGURES	viii
	LIST OF ABBREVIATIONS	ix
1.	INTRODUCTION	1
	1.1 GENERAL	1
	1.2 PROBLEM STATEMENT	2
	1.3 OBJECTIVES	3
	1.4 EXISTING SYSTEM	4
	1.5 PROPOSED SYSTEM	4
2.	LITERATURE SURVEY	6
	2.1 LITERATURE REVIEW	6
	2.1.1 FinOps and Cloud Cost Management	6
	2.1.2 Multi-Cloud Cost Management	6
	2.1.3 Cloud Cost Allocation Methods	7
	2.1.4 Resource Tagging and Cost Attribution	7
	2.1.5 Showback and Chargeback Models	8
	2.1.6 Cloud Cost Forecasting and Anomaly Detection	8
	2.1.7 Cloud Cost Optimization Techniques	8
	2.1.8 FinOps Maturity and Cloud Cost Governance	9
	2.2 COMPARISON AND DISCUSSION	10
	2.3 CONCLUSION	11
3.	SYSTEM DESIGN	12
	3.1 SYSTEM ARCHITECTURE	12
	3.2 SYSTEM REQUIREMENTS	13
	3.2.1 Software Requirements	13
	3.2.2 Hardware Requirements	14
	3.3 DATA AND PERSISTENCE DESIGN	14

	3.4 SYSTEM WORKFLOW	15
	3.4.1 Billing Data Import	15
	3.4.2 Cost Allocation and Ownership Mapping	15
	3.4.3 Showback and Chargeback Generation	16
	3.5 FINOPS COST ALLOCATION DESIGN	18
	3.6 API ARCHITECTURE	19
4.	PROJECT DESCRIPTION	20
	4.1 METHODOLOGY	20
	4.2 SYSTEM MODULES	21
	4.3 DETAILED BILLING DATA MANAGEMENT	22
	4.3.1 User Login and Access Management	22
	4.3.2 Billing Data Import	23
	4.3.3 Billing Data Validation	23
	4.3.4 Data Normalization and Duplicate Handling	24
	4.3.5 Resource Ownership Mapping	25
	4.3.6 Cost Allocation and Reporting	26
5.	IMPLEMENTATION AND RESULT DISCUSSION	28
	5.1 IMPLEMENTATION	28
	5.2 EXPERIMENTAL SETUP	30
	5.3 BUILDING DATA TEST SCENARIOS	31
	5.4 VALIDATION AND NORMALIZATION RESULTS	31
	5.5 SYSTEM FUNCTIONAL VALIDATION	32
	5.6 COST ALLOCATION AND REPORTING EVALUATION	33
	5.7 DISCUSSION	34
	5.8 LIMITATIONS	35
6.	CONCLUSION AND FUTURE WORK	36
	6.1 CONCLUSION	36
	6.2 FUTURE WORK	37
	APPENDIX 1 – BILLING DATA MODEL AND VALIDATION RULES	38
	APPENDIX 2 – DETAILED TEST CHECKLIST	39
	REFERENCES	40

LIST OF TABLES

Table 2.1 Comparison of Related Approaches	10
Table 3.1 Software Requirements	13
Table 3.2 Hardware Requirements.....	14
Table 3.3 Core Persistence Entities.....	14
Table 3.4 Semantic Relationship Taxonomy and Action Policy.....	18
Table 3.5 Major Inspection API Operations	19
Table 5.1 Test Environment Configuration	30
Table 5.2 Collaboration and Offline Test Scenarios	31
Table 5.3 Scalability Test Results from the Existing Draft	32
Table 5.4 Inspection Workflow Functional Validation	32
Table 5.5 Approved-Paper Semantic Triage Evaluation	34
Table A.1 Inspection Rules Implemented in Source Code	38
Table B.1 End-to-End Project Verification Checklist	39

LIST OF FIGURES

Figure 3.1 Overall System Architecture	13
Figure 3.2 Inspection Collection and Semantic Review Workflow	16
Figure 3.3 Inspection Finalization and PDF Export Gate.....	17
Figure 3.4 Research Semantic-Triage Cascade from the Approved Paper.....	18
Figure 4.1 Inspection Workspace with Location-Based Observations.....	24
Figure 4.2 Complementary Semantic Analysis and Suggested Combined Text.....	24
Figure 4.3 Potential Contradiction Requiring Human Review	25
Figure 4.4 Contradictory Observations Retained by Human Decision	25
Figure 4.5 Complete Multi-User Finalization with PDF Export Enabled	26
Figure 4.6 Sample Exported Inspection PDF.....	27
Figure 5.1 Dashboard Interface.....	28
Figure 5.2 Collaborative Rich-Text Editor Interface	29
Figure 5.3 AI-Generated Change Summary Panel	29
Figure 5.4 Diagnostics Panel.....	30

LIST OF ABBREVIATIONS

ABBREVIATION	EXPANSION
API	Application Programming Interface
AWS	Amazon Web Services
CSV	Comma Separated Values
DB	Database
ETL	Extract, Transform and Load
FinOps	Financial Operations
FOCUS	FinOps Open Cost and Usage Specification
GCP	Google Cloud Platform
IAM	Identity and Access Management
KPI	Key Performance Indicator
REST	Representational State Transfer
SaaS	Software as a Service
UI	User Interface
VM	Virtual Machine
Azure	Microsoft Azure
CSP	Cloud Service Provider
TCO	Total Cost of Ownership

CHAPTER 1

INTRODUCTION

1.1 GENERAL

Cloud computing has transformed the way organizations use and manage IT infrastructure by providing scalable, flexible, and on-demand computing resources. Instead of maintaining physical infrastructure, organizations can use cloud services based on their actual resource consumption. However, the use of multiple cloud providers and shared resources across departments can make cloud expenditure difficult to monitor and manage.

FinOps (Financial Operations) provides a collaborative approach to managing cloud costs by bringing together finance, technology, and business teams. It focuses on improving cost visibility, accountability, allocation, and optimization of cloud resources. Effective cost allocation helps organizations identify which departments or projects are responsible for cloud expenditure and supports informed financial decisions.

The proposed project develops a web-based FinOps framework for processing cloud billing data and improving cost transparency in multi-cloud environments. The system focuses on billing data import, validation, normalization, resource ownership mapping, cost allocation, and showback/chargeback management. It aims to provide centralized visibility into cloud expenditure while supporting efficient utilization of cloud resources.

1.2 PROBLEM STATEMENT

The rapid adoption of cloud computing has enabled organizations to use flexible and scalable infrastructure without maintaining large physical systems. However, when multiple cloud providers and shared resources are used across different departments and projects, managing cloud expenditure becomes difficult. Consolidated billing information often contains costs from different services, resources, and projects, making it challenging to identify which department or project is responsible for the expenditure. This lack of visibility can make effective cloud cost management and financial planning difficult.

A major challenge is the accurate allocation of cloud costs. Some resources can be directly associated with a particular department or project, while shared resources may be used by multiple organizational units. Inconsistent resource tagging, incomplete ownership information, duplicate records, missing values, and different billing formats can further complicate cost attribution. Organizations also require effective showback and chargeback mechanisms to provide departments with visibility into their cloud usage and associated costs. Without a centralized system, these activities may depend on manual processing, increasing the possibility of errors and reducing accountability.

Therefore, there is a need for a centralized FinOps-based web platform that can process cloud billing information and improve cost transparency in multi-cloud environments. The proposed system addresses this need through billing data import, validation and normalization, resource ownership mapping, cost allocation, and showback/chargeback management. By organizing these activities into a structured workflow, the system aims to improve visibility into cloud expenditure, strengthen departmental accountability, reduce manual processing, and support more efficient utilization of cloud resources.

1.3 OBJECTIVES

The primary objective of the proposed project is to develop a FinOps-based web platform that improves visibility and accountability of cloud expenditure in shared and multi-cloud environments. The system aims to provide a centralized approach for importing cloud billing data and organizing expenditure according to departments, projects, and resource ownership. This helps organizations understand how cloud resources are being utilized and where the associated costs are generated.

Another objective is to automate the processing and allocation of cloud costs by validating and normalizing billing data, identifying resource ownership, and supporting the allocation of both direct and shared costs. The system also aims to provide structured showback and chargeback information so that departments can clearly understand their cloud consumption and associated expenditure.

The final objective is to establish a foundation for better cloud financial management and cost optimization. By providing organized billing information, cost reports, and dashboard-based visibility, the system aims to reduce manual processing, improve departmental accountability, and support informed decisions regarding cloud resource utilization. Future enhancements will extend the platform with cloud API integration, budget forecasting, anomaly detection, and advanced cost optimization capabilities.

1.4 EXISTING SYSTEM

In the existing approach, cloud cost management is generally performed using the native billing and cost management tools provided by individual cloud service providers. These tools allow organizations to view billing information, monitor resource usage, and analyze expenditure within a particular cloud environment. However, when an organization uses multiple cloud providers, the billing information is often available separately, making it difficult to obtain a centralized view of overall cloud expenditure.

Cost allocation in existing systems often depends on resource tags, account structures, projects, or manually maintained ownership information. If resources are not consistently tagged or assigned, identifying the department or project responsible for a particular cost becomes difficult. Shared resources also require additional allocation methods, which may not always be available in a simple or consistent form.

Another limitation is the manual processing of billing data when organizations need customized departmental or project-level reports. Data may need to be exported, cleaned, organized, and analyzed before it can be used for internal reporting. This can increase processing time and the possibility of errors, particularly when billing data comes from different cloud providers.

Existing systems may also provide separate cost reports or dashboards, but they may not provide a single workflow covering billing data validation, normalization, ownership mapping, cost allocation, and showback/chargeback management. Therefore, organizations may require additional manual processes or separate tools to achieve detailed cost accountability across shared and multi-cloud environments.

1.5 PROPOSED SYSTEM

The proposed system is a web-based FinOps platform designed to improve cloud cost visibility, allocation, and accountability in shared and multi-cloud environments. It provides a centralized platform for managing cloud billing information instead of depending on separate billing reports and manual processing. The system accepts exported cloud billing data in CSV format and processes the information through a structured workflow.

The proposed workflow consists of Billing Data, Validation, Cost Allocation, Showback/Chargeback, and Dashboard. Initially, the billing data is imported into the system and validated to identify missing, duplicate, or inconsistent records. The data is then normalized into a suitable format for further processing. Resource ownership information is used to associate cloud resources with the appropriate departments or projects.

The system is designed to support both direct and shared cost allocation. Direct costs can be assigned to the department or project responsible for a resource, while shared costs can be distributed using predefined allocation rules. The resulting information can be used to generate department-wise and project-wise cost reports, providing better visibility into cloud expenditure and resource consumption.

The current implementation focuses on user login and access management, billing data import, validation, and normalization. Resource ownership mapping, cost allocation, showback, and dashboard capabilities form the subsequent development stages. Future enhancements include direct cloud API integration, budget forecasting, anomaly detection, advanced analytics, and cloud cost optimization. Overall, the proposed system aims to reduce manual cost processing, improve financial accountability, and provide a structured foundation for efficient cloud cost management.

CHAPTER 2

LITERATURE SURVEY

2.1.1 FINOPS AND CLOUD COST MANAGEMENT

FinOps, or Financial Operations, is a practice that brings together finance, technology, and business teams to manage cloud expenditure effectively. As organizations increasingly adopt cloud services, understanding how resources are being consumed and how much they cost has become important. FinOps focuses on improving cost visibility, accountability, and financial awareness by connecting cloud usage with its associated expenditure. It encourages organizations to continuously monitor their cloud spending and make informed decisions about resource utilization.

Cloud cost management involves collecting billing information, analyzing resource usage, allocating costs, and identifying opportunities to control unnecessary expenditure. In a FinOps environment, costs can be associated with departments, projects, or individual resources to provide clearer financial accountability. Showback and chargeback approaches can further help organizations communicate or assign cloud costs to responsible teams. These practices provide the foundation for the proposed system, which focuses on centralized billing data processing, cost allocation, and improved visibility of cloud expenditure.

2.1.2 MULTI-CLOUD COST MANAGEMENT

Multi-cloud cost management focuses on monitoring and controlling expenses when an organization uses services from multiple cloud providers such as AWS, Microsoft Azure, and Google Cloud. Managing costs across different platforms can be challenging because each provider has its own pricing models, billing formats, and cost-monitoring tools. A centralized approach helps organizations collect billing information from different cloud environments, organize the data, and understand how much each department, project, or resource contributes to the overall cloud expenditure. This improves cost visibility .

Effective multi-cloud cost management also involves identifying unnecessary spending, tracking resource usage, and allocating shared costs among different departments or projects. Traditional cloud billing tools generally focus on individual providers, making it difficult to obtain a unified view of expenses across multiple platforms. A centralized FinOps-based system can address this limitation by bringing billing data into a common format and providing reports and dashboards for analysis. This approach improves cost accountability, supports budgeting, and helps organizations continuously monitor and optimize their cloud spending.

2.1.3 Cloud Cost Allocation and Showback/Chargeback

Cloud cost allocation is the process of distributing cloud expenses among different departments, projects, teams, or business units based on their actual resource usage. In organizations where multiple teams share cloud resources, it can be difficult to identify who is responsible for a particular portion of the total cost. Cost allocation helps solve this problem by using factors such as resource ownership, usage, department, project, or predefined allocation rules. This provides better visibility into cloud spending and helps organizations understand where their money is being spent.

Showback and chargeback are commonly used to improve financial accountability in cloud environments. Showback provides departments with reports showing their cloud usage and associated costs without actually charging them, while chargeback assigns those costs to the responsible department or project for financial accounting. These practices encourage teams to monitor their usage and become more aware of unnecessary cloud spending. A centralized FinOps platform can support these processes by automatically allocating costs, generating departmental reports, and providing clear dashboards for tracking and managing cloud expenditure.

2.1.4 Resource Tagging and Cost Attribution

Resource tagging is a simple but important method used to keep track of cloud resources and their associated costs. Tags are labels assigned to resources such as virtual machines, databases, storage, and applications. These labels can contain details such as the department, project, owner, environment, or cost center. When resources are properly tagged, it becomes easier to identify who is using a particular resource and understand where the cloud money is being spent. This makes billing information easier to organize and analyze.

Cost attribution uses the information from these tags to assign cloud expenses to the correct department, project, or team. However, missing or incorrect tags can make cost allocation difficult and may result in some expenses remaining unassigned. Therefore, tag validation is an important part of the FinOps process, as it helps identify missing, invalid, or inconsistent resource information before allocating costs. Proper resource tagging and cost attribution improve cost visibility and accountability, and provide a reliable basis for generating showback, chargeback, and cost reports.

2.1.5 Showback and Chargeback Models

Showback and chargeback are two common methods used to improve visibility and accountability in cloud cost management. Showback provides departments or project teams with information about the cloud resources they have used and the costs associated with them, without directly charging the department. This helps teams understand their spending patterns and become more aware of how their cloud usage affects the organization's overall expenditure. Clear and understandable cost reports make it easier for teams to identify areas where cloud resources can be used more efficiently.

Chargeback, on the other hand, assigns the actual cloud costs to the department, project, or business unit responsible for the usage. It creates a stronger connection between resource consumption and financial responsibility. For example, if

two departments share cloud resources, the total cost can be divided based on their usage or predefined allocation rules. A FinOps system can support both showback and chargeback by using validated billing data, resource ownership information, and cost allocation rules to generate department-wise reports. This improves transparency, encourages responsible cloud usage, and supports better financial planning.

2.1.6 Cloud Cost Forecasting and Anomaly Detection

Cloud cost forecasting helps organizations estimate their future cloud expenses by studying previous billing and usage patterns. As cloud usage changes over time, expenses can increase due to higher resource consumption, new services, or changes in workloads. By analyzing historical cost data, organizations can identify spending trends and prepare suitable budgets for upcoming periods. Forecasting gives teams a better understanding of expected expenses and helps them take timely action when spending starts to move beyond the planned budget.

Anomaly detection complements forecasting by identifying unusual changes in cloud spending or resource usage. For example, a sudden increase in the cost of a virtual machine or storage service may indicate unexpected usage or a configuration issue. Detecting such changes early allows the responsible team to investigate the reason and take corrective action. Integrating forecasting and anomaly detection into a FinOps system provides continuous visibility into cloud spending and helps organizations manage their budgets more effectively while reducing the risk of unexpected costs.

2.1.7 Cloud Cost Optimization Techniques

Cloud cost optimization focuses on using cloud resources efficiently while avoiding unnecessary expenses. As organizations use more cloud services, they may sometimes continue paying for unused resources, choose larger resources than required, or allow resources to remain active when they are not needed. Regularly reviewing resource usage and billing information helps identify such situations. Common practices include removing unused resources, adjusting resource sizes based on actual usage, selecting suitable pricing plans, and monitoring services that contribute significantly to the overall

Cost optimization is also closely connected with continuous monitoring and informed decision-making. Instead of reducing costs at the expense of performance, organizations need to find a suitable balance between resource usage, application requirements, and expenditure. A FinOps platform can support this process by providing cost reports, identifying high-spending areas, and helping teams understand their usage patterns. These insights allow departments to take practical steps to control cloud spending, improve resource utilization, and maintain better financial control over their cloud environment.

2.1.8 FinOps Maturity and Cloud Cost Governance

FinOps maturity refers to how effectively an organization manages and controls its cloud spending as its use of cloud services grows. An organization may begin with basic activities such as monitoring bills and identifying major expenses, and gradually move towards more organized practices such as cost allocation, budgeting, forecasting, and continuous optimization. As the organization becomes more mature in FinOps, teams from finance, technology, and business departments work together to understand cloud costs and make informed decisions. This helps make cost management a regular part of cloud operations rather than a task performed only when expenses become high.

Cloud cost governance provides the rules and processes needed to maintain consistent control over cloud spending. It can include setting budgets, defining resource tagging standards, assigning ownership, monitoring spending limits, and regularly reviewing cost reports. Clear governance helps ensure that cloud resources are used responsibly and that costs can be traced back to the appropriate department or project. A FinOps platform can support these practices by providing centralized cost information, validation, allocation, reporting, and monitoring features, helping organizations maintain better visibility and accountability as their cloud environment grows.

2.2 COMPARISON AND DISCUSSION

Table 2.1: Comparison of Cloud Cost Management Approaches

Approach	Main Focus	Advantages	Limitations
FinOps	Managing cloud costs collaboratively	Improves cost visibility and accountability	Requires coordination between teams
Multi-Cloud Cost Management	Managing costs across multiple providers	Provides a broader view of cloud spending	Different billing formats can be difficult to combine
Cost Allocation	Assigning costs to departments/projects	Identifies who is responsible for spending	Depends on accurate usage and ownership data
Resource Tagging	Identifying cloud resources	Makes cost tracking and attribution easier	Missing or incorrect tags affect accuracy
Showback	Reporting costs without direct billing	Improves spending awareness	Does not directly assign financial responsibility
Chargeback	Assigning actual costs to departments	Improves financial accountability	Requires reliable allocation rules
Forecasting & Anomaly Detection	Predicting and monitoring spending	Helps identify unusual costs early	Accuracy depends on historical data
Cost Optimization	Reducing unnecessary cloud spending	Improves resource utilization	Requires continuous monitoring
Proposed FinOps System	Centralized cost management	validation, allocation, reporting, monitor	Currently based on imported billing data

The comparison shows that each approach addresses a specific part of cloud financial management. However, using these practices separately can make it difficult to obtain a complete view of cloud expenditure. A centralized system can bring billing data, resource ownership, cost allocation, and reporting together, making the overall process easier to manage and understand.

Based on the reviewed approaches, the proposed system combines billing data import, validation, normalization, resource ownership mapping, cost allocation, and showback/chargeback reporting into a single workflow. This provides a structured way to track cloud expenditure and associate costs with the appropriate departments or projects. The system is therefore designed to improve cost visibility, accountability, and financial control while keeping the cost management process simple and understandable.

2.3 CONCLUSION

The literature survey shows that effective cloud cost management requires proper visibility, accurate cost allocation, resource tracking, and continuous monitoring. Existing approaches such as FinOps, multi-cloud cost management, resource tagging, showback, chargeback, forecasting, and cost optimization address different aspects of controlling cloud expenditure. However, managing these activities separately can make it difficult for organizations to get a clear and consistent view of their overall cloud costs. The reviewed studies therefore highlight the need for a centralized and structured approach that can validate billing data, identify resource ownership, allocate costs, and generate meaningful reports. Based on these observations, the proposed system brings these functions together to provide better cost transparency, accountability, and control over cloud spending.

CHAPTER 3

SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

The proposed FinOps-based cloud cost management system is designed as a centralized platform for managing and analyzing cloud billing information. The system accepts billing data from cloud environments in the form of structured files such as CSV. Users can log in to the system and upload the required billing records. The imported data contains details such as resource usage, service type, cost, department, project, and other relevant information. This provides the basic input required for further processing and cost analysis.

Once the billing information is imported, the system checks the data for missing values, incorrect entries, inconsistent formats, and duplicate records. The data is then normalized into a common structure so that it can be processed consistently. Resource ownership information is also identified using details such as department, project, resource name, and tags. Based on the ownership information and predefined allocation rules, the system calculates and assigns the appropriate cloud costs to the respective departments or projects. The processed information is maintained in the database for further use.

The final stage of the architecture focuses on presenting the processed cost information in a simple and understandable manner. The stored data is used to generate department-wise cost summaries, showback and chargeback reports, and dashboard visualizations. These outputs help users understand how cloud resources are being used and how much each department or project contributes to the overall expenditure. Overall, the architecture provides a structured and centralized approach to cloud cost management while improving cost visibility, accountability, and reporting.

FinOps Platform Architecture

Multi-Cloud Financial Operations & Cost Management

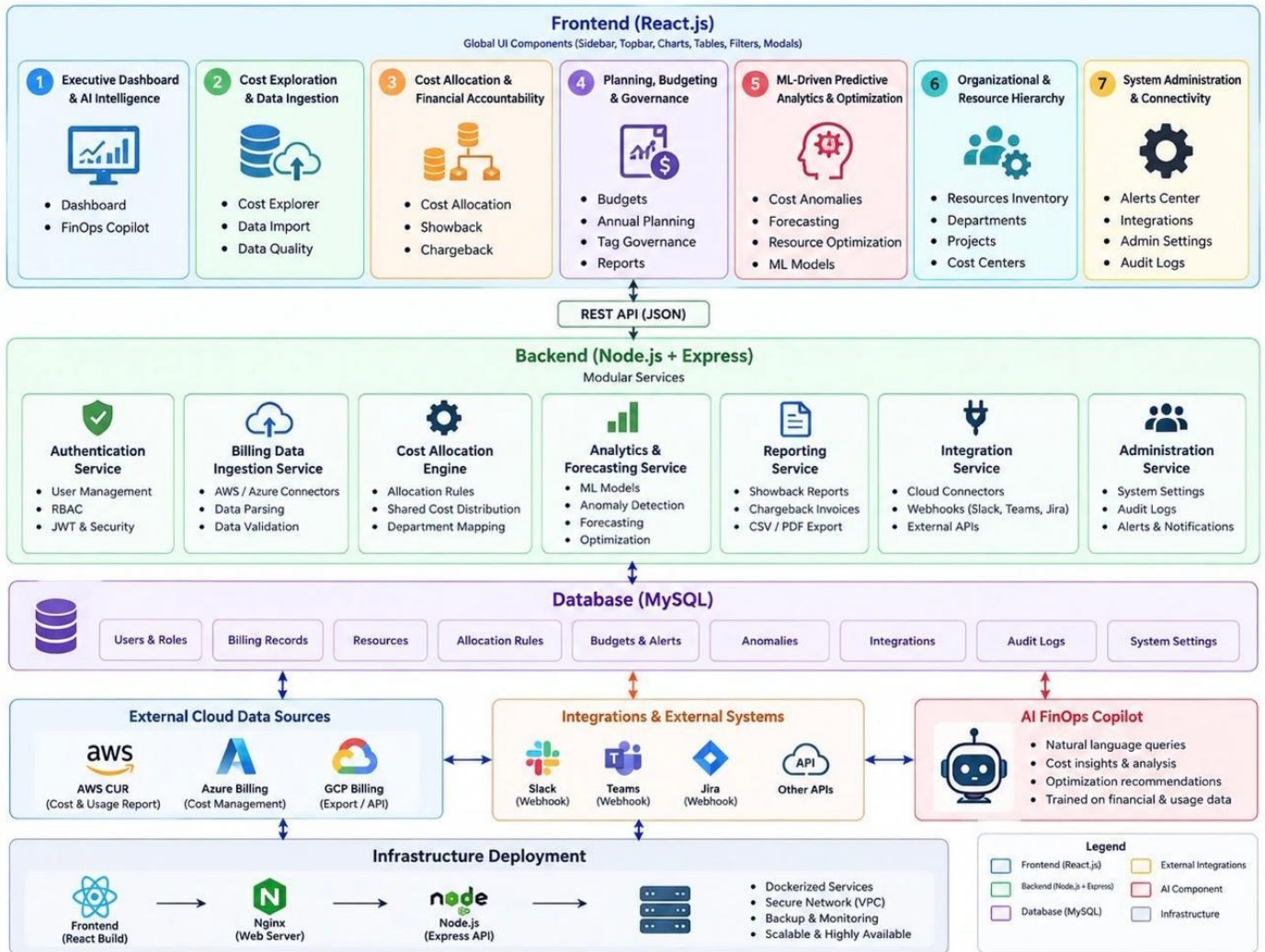


Figure 3.1 Overall System Architecture

3.2 SYSTEM REQUIREMENTS

3.2.1 Software Requirements

Table 3.2.1 Software Requirements

COMPONENT	TECHNOLOGY / REQUIREMENT
Operating System	Windows 10 / Windows 11
Frontend Development	HTML, CSS, JavaScript
Backend Development	Python with Flask
Programming Language	Python

Database	MySQL
Development Environment	Visual Studio Code
Data Processing	Pandas
Data Visualization	Chart.js
API Development	REST APIs are used to enable communication between the frontend, backend services, and database components of the application.
Web Browser	Google Chrome / Microsoft Edge

3.2.2 Hardware Requirements

Table 3.2.2 Hardware Requirements

RESOURCE	RECOMMENDED CONFIGURATION
Processor	Intel Core i5 or equivalent, 4 cores or above
RAM	8 GB minimum, 16 GB recommended
Storage	Minimum 256 GB SSD, with 512 GB recommended for storing project files, databases, datasets, and development software
Display	14-inch or larger Full HD display.
Network	Stable Wi-Fi or broadband connection.

3.3 DATA AND PERSISTENCE DESIGN

The data and persistence design defines how the FinOps system collects, organizes, and stores the information required for cloud cost management. The system handles billing records containing details such as resource name, service type, usage, cost, department, project, and resource tags. Before storing the data, the system checks the records for missing or invalid values and removes duplicate entries. The billing information is then converted into a consistent format so that it can be easily processed for cost allocation and reporting.

The processed information is stored in a structured database to ensure that it can be accessed and updated whenever required. Separate records are maintained for users, billing data, resource ownership, cost allocation, and generated reports. This

organization helps maintain data consistency and makes it easier to retrieve information for dashboards and cost analysis. The persistence design also allows previously imported billing information and allocation results to be retained, helping users track cloud expenditure and compare costs over different periods.

3.4 SYSTEM WORKFLOW

The system workflow describes the sequence of activities performed by the proposed FinOps cloud cost management system. The process begins when an authorized user logs into the application and uploads the cloud billing data in CSV format. The uploaded data is first checked for completeness and correctness. Missing values, invalid records, duplicate entries, and inconsistent formats are identified and handled during the validation and normalization process. The cleaned data is then prepared for further analysis and cost management activities.

After the data is prepared, the system identifies the ownership of cloud resources using details such as department, project, resource name, and tags. The cost allocation module then assigns the appropriate expenses to the respective departments or projects based on predefined allocation rules. The processed information is stored in the database and used to generate cost summaries, showback and chargeback reports, and dashboard visualizations. This workflow provides a systematic way to manage billing information and helps users understand and monitor their cloud expenditure.

3.4.1 Billing Data Import

The billing data import module is responsible for collecting and loading cloud billing information into the FinOps platform. The system accepts billing data in CSV format, which contains details such as service name, department, resource usage, billing period, and cost. When the file is uploaded, the system reads the data and checks whether the required fields are available before storing it for further processing.

After importing the data, the system organizes it into a structured format so that it can be used by the subsequent modules. Basic validation is performed to identify missing, invalid, or duplicate records. This ensures that the billing information used for cost allocation and reporting is reliable and consistent. The processed data is then passed to the cost allocation and data analysis modules for further operations.

3.4.2 Cost Allocation and Ownership Mapping

The cost allocation and ownership mapping module assigns cloud expenses to the appropriate departments, projects, or teams based on the available billing information. The system uses details such as department tags, project names, resource IDs, and usage information to identify who is responsible for each cloud resource. This helps avoid unassigned costs and provides a clearer view of how cloud resources are being used across the organization.

For shared resources that are used by more than one department, the system distributes the cost based on predefined allocation rules or usage values. The allocated costs are then linked to their respective owners and stored for further reporting. This process improves financial accountability and helps departments understand their actual cloud spending, which can support better budgeting and cost-control decisions.

3.4.3 Showback and Chargeback Generation

The showback and chargeback module provides a clear view of cloud expenses for each department or project. In the showback process, the system generates reports that show how much cloud resources were used and the corresponding cost, without actually transferring the cost to the department. This helps teams understand their usage and become more aware of their spending.

For chargeback, the allocated cloud costs are assigned to the respective departments or projects based on their actual usage or predefined allocation rules. The system generates detailed reports containing cost, resource usage, and ownership information. These reports help the organization maintain financial accountability and make informed decisions about cloud usage and future budgets.

3.5 FINOPS COST ALLOCATION DESIGN

The FinOps cost allocation design defines how cloud billing data is processed and distributed among different departments, projects, or teams. The design begins with importing billing data from cloud providers in CSV format. The imported data is then validated to identify missing fields, incorrect values, and duplicate records. After validation, the system identifies the owner of each cloud resource using available information such as department tags, project details, resource IDs, and usage data.

Once ownership is identified, the system calculates the cost associated with each department or project. Directly owned resources are assigned to the corresponding department, while shared resources are distributed according to predefined allocation rules or usage values. The allocated costs are then stored and used to generate showback and chargeback reports. This design provides a structured approach to tracking cloud expenditure and improves transparency and financial accountability across the organization.

Cost Allocation Process

Billing Data Import → Data Validation → Ownership Identification → Cost Allocation → Shared Cost Distribution → Showback/Chargeback → Reports & Dashboard

3.6 API ARCHITECTURE

The API architecture provides communication between the different components of the FinOps platform. It allows the frontend to send requests to the backend for operations such as uploading billing data, validating records, allocating costs, mapping resources to departments, and generating reports. The backend processes these requests and communicates with the database to store and retrieve the required information.

The architecture follows a simple request-response approach. When a user performs an action through the dashboard, the request is sent to the appropriate API endpoint. The backend validates and processes the request before returning the required result to the frontend. This approach keeps the system organized and makes it easier to maintain and extend the platform with additional cloud integrations or FinOps features in the future.

API Component Endpoint	Method	Purpose
/api/billing/import	POST	Upload and import cloud billing data in CSV format.
/api/billing/validate	POST	Validate billing records and identify missing or duplicate data.
/api/resources	GET	Retrieve cloud resource and usage information.
/api/ownership/map	POST	Map resources to the appropriate department.

/api/cost/allocation	POST	Calculate and allocate costs based on ownership and usage rules.
/api/cost/shared	POST	Distribute costs for resources shared by multiple departments.
/api/showback	GET	Generate departmental and project-wise showback information.
/api/chargeback	GET	Generate chargeback details based on allocated costs.
/api/dashboard	GET	Retrieve cost and usage data for dashboard visualization.
/api/reports	GET	Generate and retrieve FinOps cost reports.

PROJECT DESCRIPTION

4.1 METHODOLOGY

The proposed FinOps platform follows a structured methodology to manage and analyze cloud costs. The process starts with collecting cloud billing data in CSV format and importing it into the system. The imported data is then checked for missing values, duplicate records, and incorrect entries. After cleaning and validating the data, the system organizes the billing information so that it can be used for further cost analysis.

The next stage focuses on cost allocation and ownership mapping. Each cloud resource is mapped to the appropriate department or project using information such as resource IDs, tags, and usage details. Costs related to resources owned by a single department are assigned directly, while shared resource costs are distributed using predefined allocation rules. This helps the organization understand which departments or projects are responsible for the cloud expenses.

Finally, the allocated cost information is used to generate showback and chargeback reports and display the results through an interactive dashboard. The dashboard provides department-wise, project-wise, and resource-wise spending information along with cost trends. The methodology provides a systematic way to improve cost visibility, financial accountability, and future budgeting decisions while keeping the overall process simple and easy to manage.

4.2 SYSTEM MODULES

The proposed FinOps platform is divided into different modules to manage cloud billing information in a structured manner. The process begins with User Login and Access Management, which provides authorized access to the system. The Billing Data Import module allows users to upload cloud billing data in CSV format, while the Billing Data Validation module checks the uploaded records for missing, incorrect, or incomplete information.

The next set of modules focuses on improving the quality and organization of billing data. Data Normalization and Duplicate Handling converts the imported records into a consistent format and removes duplicate entries to avoid incorrect cost calculations. The Resource Ownership Mapping module then connects each cloud resource with its respective department, project, or team using details such as resource IDs, tags, and usage information.

Finally, the Cost Allocation and Reporting module processes the validated and organized billing data to calculate the cost associated with each department or project. Direct costs are assigned to the appropriate owner, while shared resource costs are distributed using predefined rules or usage values. The resulting information is used to generate showback and chargeback reports, providing a clear view of cloud expenditure and supporting better cost monitoring and financial accountability.

4.3 DETAILED BILLING DATA MANAGEMENT

The detailed billing data management module is responsible for handling cloud billing information throughout the FinOps process. It starts with secure user access and the import of billing data in CSV format. The system then checks the uploaded data for missing, invalid, or duplicate records to ensure that the information is suitable for further processing.

After validation, the billing records are normalized into a consistent format so that different types of cloud billing information can be processed easily. The system then maps each resource to its respective department, project, or owner using available resource and tagging information. This creates a clear connection between cloud resources and the teams responsible for their usage.

The processed data is finally used for cost allocation and report generation. Direct resource costs are assigned to the appropriate owner, while shared resource costs are distributed based on predefined rules or usage values. The resulting information supports showback and chargeback reports and provides users with a clear view of departmental and project-wise cloud expenditure.

4.3.1 User Login and Access Management

The user login and access management module provides secure access to the FinOps platform. Users enter their registered credentials to access the system. After successful authentication, users can access the billing, cost allocation, reporting, and dashboard features available to them. This helps protect billing information from unauthorized access.

4.3.2 Billing Data Import

The billing data import module allows users to upload cloud billing information into the system. The platform accepts billing data in CSV format containing details such as billing date, service name, resource ID, usage, department, and cost. Once uploaded, the data is read and stored for further processing.

4.3.3 Billing Data Validation

The billing data validation module checks the imported records to make sure they contain accurate and usable information. The system checks for missing fields, invalid

values, incorrect cost or usage entries, and incomplete records. Any problematic data is identified before it is used for cost allocation, helping to improve the reliability of the results.

4.3.4 Data Normalization and Duplicate Handling

The data normalization and duplicate handling module converts billing records into a common and consistent format. This makes it easier to process data from different billing sources. The system also compares records to identify duplicate entries and removes unnecessary duplicates. This prevents the same expense from being counted more than once during cost calculation.

4.3.5 Resource Ownership Mapping

The resource ownership mapping module connects each cloud resource with its responsible department, project, or team. The system uses information such as resource IDs, department tags, project names, and usage details to determine ownership. If a resource is shared by multiple departments, it is identified as a shared resource and handled separately during cost allocation.

4.3.6 Cost Allocation and Reporting

The cost allocation and reporting module assigns the processed cloud expenses to the appropriate departments or projects. Costs belonging to a single owner are directly assigned, while shared resource costs are distributed using predefined rules or usage-based values. The allocated information is then used to generate showback and chargeback reports, providing a clear view of department-wise, project-wise, and resource-wise cloud spending.

CHAPTER 5

IMPLEMENTATION AND RESULT DISCUSSION

Figure 5.4 Diagnostics Panel

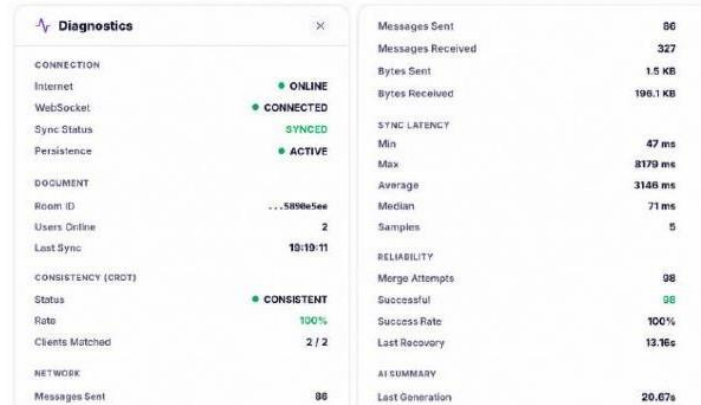


Figure 5.4 Diagnostics Panel

5.1 EXPERIMENTAL SETUP

The synchronization evaluation described in the existing draft used a local development server, desktop browser clients and a mobile browser on the local network. Wi-Fi and a mobile hotspot were used to exercise normal connectivity and interruption/recovery behavior. The recorded draft configuration used Yjs 13.6.31 on the client and 13.6.22 on the server, Google Chrome 151.0.7922.174, and a test laptop with a 13th Gen Intel Core i7-13650HX processor and 24 GB RAM. These measurements characterize the development setup and should not be interpreted as production capacity limits.

Table 5.1 Test Environment Configuration

PARAMETER	VALUE
Browsers	Google Chrome and Microsoft Edge; mobile browser over local network
Network	Wi-Fi, mobile hotspot and simulated disconnect/reconnect
Yjs versions	Client 13.6.31; server 13.6.22
Test browser	Google Chrome 151.0.7922.174
Test machine	13th Gen Intel Core i7-13650HX; 24 GB RAM
Monitoring	In-editor diagnostics: connectivity, consistency, latency and message counts

5.2 COLLABORATION TEST SCENARIOS

Table 5.2 Collaboration and Offline Test Scenarios

ID	SCENARIO	PURPOSE
T1	Simultaneous editing	Verify real-time collaborative updates
T2	Multiple concurrent edits	Verify preservation of independent changes
T3	Single-client offline editing	Verify uninterrupted local editing
T4	Multiple clients offline	Verify independent offline changes
T5	Reconnection and synchronization	Verify recovery and state convergence
T6	Browser refresh while offline	Verify IndexedDB persistence
T7	Close and reopen while offline	Verify persisted restoration
T8	Network interruption during synchronization	Verify repeated recovery
T9	Concurrent editing in the same area	Observe CRDT conflict handling
T10	Increasing concurrent users	Observe behavior at 2/5/10/20 simulated users
T11	Different document rooms	Verify room isolation

5.3 SYNCHRONIZATION RESULTS

Under normal connectivity, edits were reflected on collaborating clients without a page refresh. The diagnostics panel reported the document state as consistent and the draft records a median synchronization latency of 71 ms during the test session. Across the evaluated offline/reconnection scenarios, 98 merge attempts were recorded and all completed successfully, corresponding to a 100% merge success rate in the reported test set.

Offline persistence was verified by continuing to edit a disconnected client, refreshing the browser while offline and closing/reopening the document before reconnection. Locally entered content remained available. When connectivity returned, offline and remote changes were merged and the diagnostics status returned to synchronized/consistent. A separate room-isolation test confirmed that updates from one document room did not appear in another.

The scalability script tested 2, 5, 10 and 20 simulated users. The measured averages decreased from 44.51 ms at two users to 12.68 ms at twenty users. The draft correctly treats this as an observed characteristic of the local test configuration rather than evidence that adding users improves latency.

Table 5.3 Scalability Test Results from the Existing Draft

CONCURRENT USERS	AVG. SYNC LATENCY	MESSAGES SENT	MESSAGES RECEIVED	DATA SENT	DATA RECEIVED
2	44.51 ms	48	53	1.21 KB	3.74 KB
5	29.74 ms	120	505	3.02 KB	24.53 KB
10	18.52 ms	240	2,225	6.04 KB	96.96 KB
20	12.68 ms	480	9,179	12.29 KB	465.83 KB

5.4 INSPECTION FEATURE FUNCTIONAL VALIDATION

Table 5.4 Inspection Workflow Functional Validation

ID	TEST	EXPECTED BEHAVIOR	EVIDENCE / RESULT
I1	Create inspection and location	Location appears in the location tree	Supported by implemented CRUD and captured UI
I2	Two collaborators add observations	Both authored cards shown with online/offline state	Observed in supplied screenshots
I3	Complementary observations	Relationship, confidence, explanation and combined text displayed	Observed: 92% complementary example
I4	Contradictory observations	Potential contradiction displayed without choosing factual winner	Observed: 95% contradiction example
I5	Human resolution	Reviewer decision persisted and status reflected	Observed “KEPT BOTH” state; controller persists decision metadata
I6	Partial finalization	Finalization count updates but PDF stays locked	Observed 1/2 Finalized with export locked
I7	All users finalized	Inspection becomes finalized and export becomes available	Observed 2/2 Finalized with Export PDF enabled
I8	PDF generation	Export groups observations by location and includes resolution	Confirmed by supplied sample export PDF

The captured A Block example demonstrates the complementary path. Two collaborators contribute related descriptions of a crack in Room 101. The

semantic panel classifies the pair as complementary, explains that one statement contributes exact location and the other contributes severity, and proposes a combined sentence. This matches the implemented COMBINE action and shows how AI assistance can improve completeness without discarding either observation.

The B Block example demonstrates the safety-critical contradiction path. One collaborator states that the entrance fan is broken, while another states that the entrance fan is fine and a different fan is damaged. The interface reports a potential contradiction with 95% confidence and explains the incompatibility. The user retains both observations, and that human outcome is later represented in the exported PDF. The system therefore exposes disagreement instead of silently rewriting it.

The finalization screenshots validate the approval gate. At 1/2 finalized, report status shows PDF Export as locked. At 2/2 finalized, the workspace is marked FINALIZED, the status message indicates that all collaborators have finalized and the Export PDF control becomes available. This behavior corresponds directly to the backend report-status and finalizeReport logic.

5.5 SEMANTIC RESEARCH EVALUATION FROM THE APPROVED PAPER

The approved conference paper evaluates the broader semantic-triage framework on a separate 300-pair, two-annotator benchmark. That evaluation is not identical to the deployed Gemini-only Inspection service and is therefore reported separately rather than being presented as a production metric. The paper’s full-cascade evaluation reported Cohen’s kappa of 0.800, 80.7% LLM-avoidance coverage, 87.3% contradiction recall and 75.7% four-class classification accuracy.

The contradiction-recall result is particularly relevant to the project’s policy because missed contradictions are more consequential than unnecessary

review prompts. The research design therefore favors conservative escalation. The implemented Inspection feature follows the same governance principle even though its current inference path is simpler: contradictions remain human-review items and the model is prohibited from selecting the “true” inspector statement.

Table 5.5 Approved-Paper Semantic Triage Evaluation

METRIC	REPORTED VALUE	INTERPRETATION
Two-annotator agreement (Cohen’s kappa)	0.800	Substantial agreement in the 300-pair evaluation
Classification accuracy	75.7%	Overall four-class performance of the evaluated cascade
Contradiction recall	87.3%	Sensitivity to the safety-critical contradiction class
LLM-avoidance coverage	80.7%	Share of pairs handled without LLM use in the paper’s cascade

5.6 DISCUSSION

The results support the architectural separation used by the project. Yjs solves the availability and convergence problem, while the semantic layer addresses a different class of content risk after observations have been collected. Because the semantic service cannot modify Yjs merge semantics, a temporary AI failure does not prevent users from synchronizing their work. The controller also continues with remaining observation pairs if one AI call fails.

The Inspection workflow adds governance that a general editor does not normally provide. Semantic decisions are durable records, finalization is user-specific, and export is a collective action. This is important in reports where collaborators may be professionally accountable for their observations. The sample PDF shows that unresolved factual disagreement can remain visible as a retained conflict rather than being obscured by generated prose.

The principal limitation is that the final implementation does not yet deploy every component of the research paper’s lightweight cascade. The production semantic service currently relies on pair filtering plus Gemini, so its cost, calibration and generalization must be assessed separately from the approved paper’s SBERT/NLI/LLM evaluation. The current report therefore keeps research results and implementation validation distinct.

5.7 LIMITATIONS

- The synchronization measurements come from a development environment and do not establish production-scale capacity.
- The highest reported load-test concurrency was 20 simulated users; longer-running sessions and larger documents were not exhaustively evaluated.
- Network evaluation focused on disconnection/reconnection and did not systematically characterize sustained packet loss, extreme latency or prolonged low-bandwidth conditions.
- The implemented Inspection semantic service depends on an external Gemini API and may vary across model versions, prompts and service availability.
- Confidence returned by the model is an application aid rather than a statistically calibrated probability unless separately calibrated on domain data.
- Pairwise comparison is location-local and may miss long-range semantic dependencies between observations in different locations or across a long report.
- The supplied functional screenshots demonstrate representative Inspection behavior; a larger user study is still required to evaluate usability and review workload in real field inspections.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

This project designed and implemented an offline-first collaborative platform in which document availability and replica convergence are handled by Yjs CRDTs, while application-level semantics remain a separate concern. Local IndexedDB persistence enables continued editing during disconnection, WebSocket synchronization restores shared state after reconnection, and MongoDB provides server-side persistence and queryable application data.

The major final-stage addition is the Inspection workspace. It converts the semantic-conflict concept from the approved paper into a concrete collaborative workflow built around locations, authored observations, four-class semantic analysis, human decisions, participant finalization and controlled PDF export. The system does not allow the AI to declare which inspector is factually correct. Instead, it classifies the relationship, explains its reasoning, suggests a minimally invasive action and preserves human authority over contradictions.

The implemented finalization rule further strengthens accountability: semantic review must be complete before users can finalize, and the report cannot be exported until all required participants have finalized. The sample export confirms that the resulting PDF can preserve both observations and the selected resolution. Within the evaluated development environment, the existing project tests also reported reliable offline recovery, 100% recorded merge success and consistent final replicas. The project therefore demonstrates how CRDT synchronization, AI assistance and human review can coexist without conflating state convergence with semantic correctness.

6.2 FUTURE WORK

- Integrate the full lightweight cascade described in the approved paper, using structural filtering, Sentence-BERT similarity and bidirectional NLI before selective LLM escalation.
- Calibrate confidence and routing thresholds using a larger inspection-specific benchmark rather than relying only on general-language examples.
- Add richer evidence handling for factual disputes, including photographs, annotations, measurements, timestamps and links to inspection standards.
- Extend semantic analysis from pairwise location-local observations to longer dependency chains and cross-location consistency checks.
- Provide configurable roles for inspector, reviewer and approving authority, with digital signatures and immutable audit export where required.
- Perform field studies with real inspection teams to measure review workload, user trust, false-positive burden and time-to-final-report.
- Expand scalability and network testing to larger user counts, larger documents, longer offline intervals, packet loss and variable bandwidth.
- Explore on-device or privacy-preserving semantic models for deployments where inspection text cannot be sent to a third-party model service.
- Investigate hybrid or peer-to-peer synchronization to reduce dependence on a central WebSocket relay while preserving persistence and access control.

APPENDIX 1

INSPECTION DATA MODEL AND STATUS RULES

This appendix summarizes implementation details extracted from the final project source code. It is included to document the exact final-stage Inspection behavior without overloading the main chapters.

Table A.1 Inspection Rules Implemented in Source Code

AREA	IMPLEMENTED RULE
Inspection status	DRAFT, SYNCHRONIZING, REVIEW_REQUIRED, READY_TO_FINALIZE, FINALIZED
Observation connectivity	online or offline stored with observation snapshot
Semantic relationship	DUPLICATE, COMPLEMENTARY, CONTRADICTORY, INDEPENDENT
Human decision	PENDING, ACCEPTED, REJECTED, KEPT_BOTH, KEPT_ONE, EDITED
Candidate filtering	Observation text must contain at least five trimmed characters
AI retry	Up to five retries for HTTP 503 with exponential backoff
Finalization prerequisite	No PENDING non-independent semantic result
Complete finalization	Owner plus all unique collaborators recorded in finalizedBy
PDF availability	Only when inspection is effectively finalized
Export contents	Inspection metadata, observations and semantic results with non-pending decisions

APPENDIX 2

DETAILED TEST CHECKLIST

Table B.1 End-to-End Project Verification Checklist

ID	AREA	CHECK
B1	Authentication	Register/login and open protected dashboard
B2	Document creation	Create a new collaborative document
B3	Real-time edit	Open same document in two clients and verify remote update
B4	Offline edit	Disconnect one client and continue typing
B5	IndexedDB persistence	Refresh/close while offline and verify local content remains
B6	Reconnection	Restore network and verify CRDT convergence
B7	Room isolation	Edit two independent document rooms and verify no cross-room updates
B8	Inspection creation	Create an inspection and verify owner access
B9	Share inspection	Invite collaborator with editor/viewer permission
B10	Location creation	Create multiple inspection locations
B11	Observation snapshot	Add observations and verify stored author/connectivity metadata
B12	Complementary classification	Create compatible observations and verify combined text suggestion
B13	Contradiction classification	Create incompatible observations and verify human-review prompt
B14	Human decision	Record accepted/kept-both/other resolution and verify persistence
B15	Review gate	Leave a required result pending and verify finalization is blocked
B16	Partial finalization	Finalize one user and verify PDF export remains locked
B17	Complete finalization	Finalize all users and verify FINALIZED status
B18	PDF export	Export and verify locations, observations and resolved conflicts appear
B19	Deletion cleanup	Delete observation and verify related stale semantic results are removed
B20	Research log	Verify analysis/decision/finalization events are logged

REFERENCES

- [1] C. Sun and C. A. Ellis, “Operational transformation in real-time group editors: issues, algorithms, and achievements,” in Proc. ACM Conf. Computer Supported Cooperative Work (CSCW), 1998, pp. 59-68. doi: 10.1145/289444.289469.
- [2] N. Preguica, J. M. Marques, M. Shapiro, and M. Letia, “A commutative replicated data type for cooperative editing,” in Proc. 29th IEEE Int. Conf. Distributed Computing Systems (ICDCS), 2009, pp. 395-403. doi: 10.1109/ICDCS.2009.20.
- [3] M. Shapiro, N. Preguica, C. Baquero, and M. Zawirski, “Conflict-free replicated data types,” in Proc. 13th Int. Conf. Stabilization, Safety, and Security of Distributed Systems (SSS), 2011, pp. 386-400. doi: 10.1007/978-3-642-24550-3_29.
- [4] M. Kleppmann and A. R. Beresford, “A conflict-free replicated JSON datatype,” IEEE Trans. Parallel Distrib. Syst., vol. 28, no. 10, pp. 2733-2746, 2017. doi: 10.1109/TPDS.2017.2697382.
- [5] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan, “Local-first software: You own your data, in spite of the cloud,” in Proc. ACM SIGPLAN Int. Symp. New Ideas, New Paradigms, and Reflections on Programming Software (Onward!), 2019, pp. 154-178. doi: 10.1145/3359591.3359737.
- [6] N. Schiefer, G. Litt, and D. Jackson, “Merge what you can, fork what you can’t: managing data integrity in local-first software,” in Proc. 9th Workshop on Principles and Practice of Consistency for Distributed Data (PaPoC), 2022, pp. 24-32. doi: 10.1145/3517209.3524041.
- [7] G. Litt, S. Lim, M. Kleppmann, and P. van Hardenberg, “Peritext: A CRDT for collaborative rich text editing,” Proc. ACM Hum.-Comput. Interact., vol. 6, no. CSCW2, Art. 531, 2022. doi: 10.1145/3555644.
- [8] J. Gentle and M. Kleppmann, “Collaborative text editing with Eg-walker: Better, faster, smaller,” in Proc. 20th European Conf. Computer Systems (EuroSys), 2025, pp. 311-328. doi: 10.1145/3689031.3696076.
- [9] M. Weidner, J. Gentle, and M. Kleppmann, “The art of the Fugue: Minimizing interleaving in collaborative text editing,” IEEE Trans. Parallel Distrib. Syst., vol. 36, no. 11, pp. 2425-2437, 2025. doi: 10.1109/TPDS.2025.3611880.
- [10] G. Semenov and V. Aksenov, “Semantic conflict model for collaborative data structures,” in Proc. 13th Int. Workshop on Principles and Practice of Consistency for Distributed Data (PaPoC), 2026, pp. 25-31. doi: 10.1145/3806077.3806700.
- [11] N. Gu, J. Xu, X. Wu, J. Yang, and W. Ye, “Ontology based semantic conflicts resolution in collaborative editing of design documents,” Advanced Engineering Informatics, vol. 19, no. 2, pp. 103-111, 2005. doi: 10.1016/j.aei.2005.05.005.
- [12] D. Yang, A. Halfaker, R. Kraut, and E. Hovy, “Identifying semantic edit intentions from revisions in Wikipedia,” in Proc. 2017 Conf. Empirical Methods in Natural Language Processing (EMNLP), pp. 2000-2010. doi: 10.18653/v1/D17-1213.
- [13] S. R. Bowman, G. Angeli, C. Potts, and C. D. Manning, “A large annotated corpus for learning natural language inference,” in Proc. 2015 Conf. Empirical Methods in Natural Language Processing (EMNLP), pp. 632-642. doi: 10.18653/v1/D15-1075.

- [14] A. Williams, N. Nangia, and S. R. Bowman, “A broad-coverage challenge corpus for sentence understanding through inference,” in Proc. NAACL-HLT, 2018, pp. 1112-1122. doi: 10.18653/v1/N18-1101.
- [15] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in Proc. EMNLP-IJCNLP, 2019, pp. 3982-3992. doi: 10.18653/v1/D19-1410.
- [16] J. Thorne, A. Vlachos, C. Christodoulopoulos, and A. Mittal, “FEVER: a large-scale dataset for fact extraction and verification,” in Proc. NAACL-HLT, 2018, pp. 809-819. doi: 10.18653/v1/N18-1074.
- [17] M. Lee, P. Liang, and Q. Yang, “CoAuthor: Designing a human-AI collaborative writing dataset for exploring language model capabilities,” in Proc. CHI Conf. Human Factors in Computing Systems, 2022, Art. 388, pp. 1-19. doi: 10.1145/3491102.3502030.
- [18] Q. Ruan, I. Kuznetsov, and I. Gurevych, “Re3: A holistic framework and dataset for modeling collaborative document revision,” in Proc. 62nd Annual Meeting of the Association for Computational Linguistics (ACL), 2024, pp. 4635-4655. doi: 10.18653/v1/2024.acl-long.255.
- [19] L. Gao et al., “RARR: Researching and revising what language models say, using language models,” in Proc. 61st Annual Meeting of the Association for Computational Linguistics (ACL), 2023, pp. 16477-16508. doi: 10.18653/v1/2023.acl-long.910.
- [20] P. Manakul, A. Liusie, and M. J. F. Gales, “SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models,” in Proc. 2023 Conf. Empirical Methods in Natural Language Processing (EMNLP), pp. 9004-9017. doi: 10.18653/v1/2023.emnlp-main.557.